

Componente Database

Premessa

Questo documento descrive le componenti della parte database dell'applicazione, focalizzandosi sulla struttura, la responsabilità e l'utilizzo. E' volto al supporto della comprensione, manutenzione e futuri cambiamenti dal lato database.

Specifica requisiti e ambito

La presente sezione documenta la componente database dell'applicazione, descrivendone la struttura, le scelte progettuali e le modalità di utilizzo. Il database rappresenta il livello di persistenza dell'applicazione ed è responsabile della gestione dei dati relativi agli utenti.

L'ambito della documentazione include il modello dei dati, i vincoli di integrità, lo script di inizializzazione e le principali modalità di accesso alle informazioni. Non rientrano invece nell'ambito di questo documento le logiche applicative di frontend.

Pubblico di riferimento

La documentazione è rivolta principalmente a sviluppatori che devono comprendere, utilizzare o estendere la componente database dell'applicazione. Si assume una conoscenza di base dei database relazionali e del linguaggio SQL, ma non una conoscenza preventiva della struttura specifica del progetto.

Tecnologie utilizzate

Il database è implementato utilizzando PostgreSQL, scelto per il supporto avanzato ai vincoli di integrità, alla gestione delle transazioni e alla conformità agli standard SQL. Lo schema del database viene creato e inizializzato tramite uno script SQL dedicato (`init.sql`), eseguito in fase di setup dell'applicazione.

Analisi Requisiti

Modello concettuale (ER)

Il modello concettuale descrive le principali entità del dominio applicativo e le relazioni tra esse. L'obiettivo è rappresentare in modo astratto i dati gestiti dall'applicazione e i vincoli logici che li collegano, indipendentemente dalla loro implementazione fisica in PostgreSQL.

Entità principali

Il modello ER include le seguenti entità principali:

- **Utente**: rappresenta l'utente dell'applicazione .
- **Carriera**: rappresenta la carriera dello studente
- **Corso**: rappresenta il corso di laurea a cui un utente è associato.
- **Insegnamento**: rappresenta un insegnamento previsto all'interno di un corso.
- **Esame**: rappresenta un esame sostenuto (o registrato) da un utente, associato a uno specifico insegnamento.

Relazioni e cardinalità

Le relazioni tra le entità sono le seguenti:

- **Utente–Carriera** (*carriera_studente*): un **Utente** è associato a **una o più Carriere** (1:N), mentre ogni **Carriera** è associata a **un solo Utente**.
- **Carriera–Corso** (*corso_di_laurea_nella_carriera*): una **Carriera** può essere associata a **un Corso** (1:1).
- **Corso–Insegnamento** (*insegnamenti_del_corso_di_laurea*): un **Corso** contiene **uno o più Insegnamenti** (1:N), e ogni **Insegnamento** può appartenere a **un solo Corso**.
- **Insegnamento–Esame** (*esame_dell'insegnamento*): un **Insegnamento** può avere **uno o più Esami** (1:N), mentre ogni **Esame** è riferito a **un solo Insegnamento**.

Vincoli concettuali

A livello concettuale, lo schema implica i seguenti vincoli:

- Ogni **Carriera** deve riferirsi a un **Utente**.
- Ogni **Insegnamento** deve riferirsi a un **Corso**.
- Ogni **Esame** deve riferirsi a un **Insegnamento**.
- Ogni **Carriera** deve avere un **Corso**.

Riferimento allo schema

Lo schema ER è riportato in Figura 1.

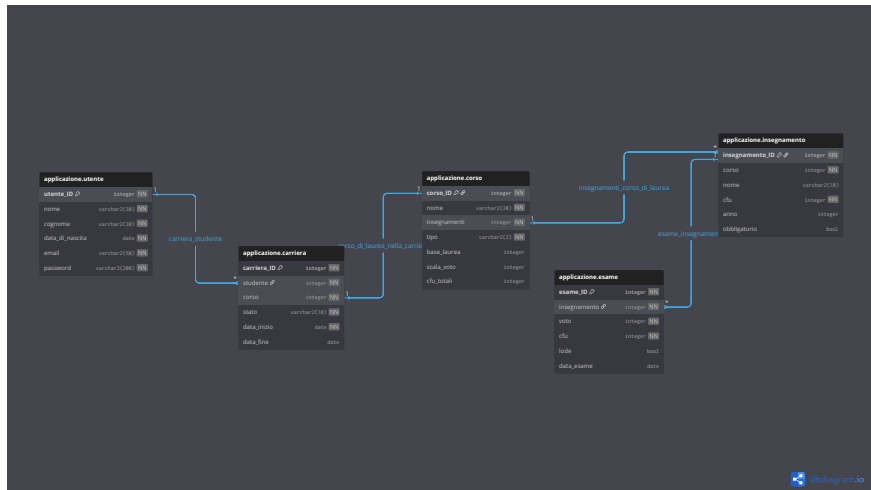


Figure 1: Schema Entity-Relationship (ER) del database.

Modello logico (schema relazionale)

Le tabelle sono organizzate nello schema `applicazione` e seguono una nomenclatura coerente con lo schema ER.

Tabella `applicazione.utente`

- **Scopo**: anagrafica e credenziali dell'utente.
- **PK**: `utente_id`
- **Attributi**:
 - `utente_id` (integer)
 - `nome` (varchar(30))
 - `cognome` (varchar(30))
 - `data_di_nascita` (date)
 - `email` (varchar(50))
 - `password` (varchar(100))

Tabella `applicazione.corso`

- **Scopo:** descrizione del corso di laurea e parametri utili per calcoli (base laurea, scala voto, CFU totali).
- **PK:** `corso_id`
- **Attributi:**
 - `corso_id` (integer)
 - `nome` (varchar(20))
 - `insegnamenti` (integer)
 - `tipo` (varchar(2))
 - `base_laurea` (integer)
 - `scala_voto` (integer)
 - `cfu_totali` (integer)

Tabella `applicazione.insegnamento`

- **Scopo:** definisce gli insegnamenti associati a un corso di laurea.
- **PK:** `insegnamento_id`
- **FK:**
 - `corso` → `applicazione.corso(corso_id)`
(relazione *Corso-Insegnamento* 1:N)
- **Attributi:**
 - `insegnamento_id` (integer)
 - `corso` (integer)
 - `nome` (varchar(10))
 - `cfu` (integer)
 - `anno` (integer)
 - `obbligatorio` (boolean)

Tabella `applicazione.esame`

- **Scopo:** registra gli appelli d'esame associati a un insegnamento.
- **PK:** `esame_id`
- **FK:**
 - `insegnamento` → `applicazione.insegnamento(insegnamento_id)` (relazione *Insegnamento-Esame* 1:N)

- **Attributi:**

- esame_id (integer)
- insegnamento (integer)
- cfu (integer)
- data_esame (date)

Tabella applicazione.carriera

- **Scopo:** rappresenta la carriera dello studente e collega utente e corso.

- **PK:** carriera_id

- **FK:**

- studente → applicazione.utente(utente_id)
(relazione *Utente-Carriera* 1:1)
- corso → applicazione.corso(corso_id)
(relazione *Carriera-Corso* 1:1)

- **Attributi:**

- carriera_id (integer)
- studente (integer)
- corso (integer)
- stato (varchar(10))
- data_inizio (date)
- data_fine (date)

Inizializzazione del database

L'applicazione utilizza `docker-compose` per orchestrare i servizi **frontend**, **backend** e **database**. Il database è implementato tramite PostgreSQL e viene avviato come servizio dedicato (**db**) all'interno dello stack.

Orchestrazione dei servizi

Il file `docker-compose.yml` definisce i seguenti servizi:

- **frontend:** web server (Nginx) che espone l'interfaccia utente su `localhost:8080`;
- **backend:** servizio PHP che accede al database tramite variabili d'ambiente;
- **db:** servizio PostgreSQL (build custom) che gestisce la persistenza dei dati.

Configurazione del database

Il servizio db inizializza PostgreSQL tramite le variabili d'ambiente: `POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`. Il servizio espone la porta 5432 verso l'host.

Esecuzione di `init.sql`

L'immagine PostgreSQL esegue automaticamente gli script presenti nella directory `/docker-entrypoint-initdb.d/`. Il file `init.sql` viene copiato in tale directory durante la build dell'immagine del servizio db. Di conseguenza, al primo avvio vengono creati schema, tabelle e vincoli in modo ripetibile e deterministico. Per garantire che prima venga creata la struttura il file copiato viene rinominato in `/docker-entrypoint-initdb.d/01_init.sql` dato che Postgres esegue i file in ordine alfabetico.

Esecuzione di `seed.sql`

Un altro file presente nella directory `/docker-entrypoint-initdb.d/` è `seed.sql` che viene copiato allo stesso modo di `init.sql` e popola il database di dati fittizi. Per garantire che `seed.sql` venga eseguito come secondo, quindi dopo la creazione della struttura, questo viene rinominato in `/docker-entrypoint-initdb.d/02_seed.sql`

Applicazione del Factory Method Pattern

Il processo di inizializzazione del database implementa il Factory Method pattern:

- Il servizio PostgreSQL funge da **Creator**, definendo l'interfaccia per eseguire script SQL
- La convenzione di naming (`01_`, `02_`) costituisce il **ConcreteCreator**, determinando l'ordine di creazione degli oggetti
- Ciascuno script produce un **ConcreteProduct** specifico (schema, tabelle, dati)
- Questo design permette di estendere il processo di inizializzazione aggiungendo nuovi script senza modificare il sistema di bootstrap

Persistenza dei dati (volume)

Il servizio db utilizza un volume Docker nominato (`postgres_data`) montato su `/var/lib/postgresql/data`. Questo volume rende persistenti i dati tra riavvii del container. Di conseguenza, `init.sql` viene eseguito **solo la prima volta**: ai riavvii successivi, PostgreSQL rileva una directory dati già inizializzata e non riesegue gli script di bootstrap.

Vincoli e integrità dei dati

La coerenza e l'affidabilità dei dati sono garantite mediante vincoli definiti direttamente a livello di database. Questo approccio riduce la dipendenza dalla logica applicativa e assicura che i dati rimangano validi anche in caso di accessi diretti al DB o evoluzioni future dell'applicazione.

Integrità di entità

Ogni tabella definisce una chiave primaria (PRIMARY KEY) che identifica in modo univoco ciascun record. Le chiavi primarie sono implementate tramite attributi numerici con generazione automatica (incrementale) del valore.

Gli attributi obbligatori sono marcati con il vincolo NOT NULL, impedendo l'inserimento di record incompleti.

Integrità referenziale

Le relazioni tra le entità sono implementate tramite vincoli di chiave esterna (FOREIGN KEY), che garantiscono la coerenza tra le tabelle correlate. In particolare:

- una **carriera** deve riferirsi a un **utente** esistente;
- un **insegnamento** deve essere associato a un **corso**;
- un **esame** deve riferirsi a un **insegnamento**.

Tali vincoli impediscono la creazione di riferimenti non validi e l'eliminazione di record ancora utilizzati, preservando l'integrità del modello.

Vincoli di unicità

Per evitare duplicazioni logiche, sono definiti vincoli UNIQUE su attributi che devono essere univoci all'interno del sistema. In particolare, l'indirizzo email dell'utente è soggetto a un vincolo di unicità, garantendo che ogni account sia associato a un solo utente.

Vincoli di dominio

I vincoli CHECK limitano il dominio dei valori ammessi per alcuni attributi, garantendo la validità semantica dei dati.

- **Età minima:** sulla tabella `applicazione.utente` è definito un vincolo che consente l'inserimento di soli utenti maggiorenni. Il vincolo verifica che la data di nascita sia antecedente di **almeno 18 anni** rispetto alla data corrente.
- **Base laurea:** sulla tabella `applicazione.corso` è definito il vincolo di base per la laurea impostato a **60**.

- **Scala voto:** sempre sulla tabella `applicazione.corso` è definito anche il convolo della scala voto impostato a **110**

Vincoli applicativi

Alcune regole di business sono implementate direttamente a livello di database per garantire coerenza globale del sistema. In particolare:

- una **carriera** deve essere sempre associata a un **utente**;
- una **carriera** deve essere sempre associata a un **corso**;
- un **insegnamento** deve appartenere a un **corso**;
- un **esame** non può esistere senza il relativo **insegnamento**.

Considerazioni progettuali

L'adozione di vincoli a livello di database consente di intercettare errori già in fase di inserimento dei dati e rende il sistema più robusto rispetto a utilizzi impropri o a modifiche future della logica applicativa.

Query principali e casi d'uso

Le operazioni di lettura/scrittura sul database sono eseguite dal backend tramite il livello di *repository*. Le query SQL riportate di seguito sono esempi delle interrogazioni eseguite internamente dalle repository, usando statement parametrizzati per evitare SQL injection.

Recupero delle informazioni di un utente

Questa query consente di ottenere i dati anagrafici di un utente a partire dal suo identificativo.

```
SELECT nome, cognome, email, data_di_nascita
FROM applicazione.utente
WHERE utente_id = :utente_id;
```

Visualizzazione della carriera di uno studente

La query recupera le informazioni relative alla carriera di un utente, includendo il corso di laurea associato (se presente).

```
SELECT c.carriera_id,
       c.stato,
       c.data_inizio,
       c.data_fine,
       co.nome AS corso
FROM applicazione.carriera c
JOIN applicazione.corso co ON c.corso = co.corso_id WHERE c.studente =
:utente_id;
```


Elenco degli insegnamenti di un corso

La query restituisce tutti gli insegnamenti associati a un determinato corso di laurea.

```
SELECT e.esame_id, e.voto, e.cfu, e.lode, e.data_esame
FROM applicazione.esame e
WHERE e.insegnamento = :insegnamento_id
ORDER BY e.data_esame DESC;
```

Recupero degli esami associati a un insegnamento

Permette di visualizzare gli esami programmati o registrati per uno specifico insegnamento.

```
SELECT co.corso_id, co.nome,
COUNT(i.insegnamento_id) AS numero_insegnamenti
FROM applicazione.corso co
LEFT JOIN applicazione.insegnamento i ON co.corso_id = i.corso
GROUP BY co.corso_id, co.nome;
```

Conteggio degli insegnamenti per corso

Query di tipo aggregato utilizzata per ottenere informazioni di sintesi a partire dai dati relazionali.

```
SELECT co.nome,
COUNT(i.insegnamento_id) AS numero_insegnamenti
FROM applicazione.corso co
LEFT JOIN applicazione.insegnamento i
ON co.corso_id = i.corso
GROUP BY co.nome;
```

Repository pattern

Il Repository Pattern è stato adottato per astrarre e incapsulare la logica di accesso ai dati, separando chiaramente la business logic dal meccanismo di persistenza. Questo pattern funge da intermediario tra il dominio dell'applicazione e il livello di mappatura dei dati, fornendo un'astrazione delle operazioni CRUD. Nel nostro sistema, il Repository Pattern è implementato attraverso classi dedicate che gestiscono l'accesso a ciascuna entità principale del database:

Ruolo delle repository

Le repository si occupano di:

- eseguire query SQL sul database;
- mappare i risultati in strutture utilizzabili dall'applicazione;

- fornire un'interfaccia coerente per l'accesso ai dati.

Repository	Entità gestita	Responsabilità
UtenteRepository	Utente	Autenticazione, registrazione, gestione profilo
CarrieraRepository	Carriera	Gestione carriere studenti, associazione corsi
CorsoRepository	Corso	Recupero informazioni corsi di laurea
InsegnamentoRepository	Insegnamento	Gestione insegnamenti per corso
EsameRepository	Esame	Registrazione e consultazione esami

Il resto dell'applicazione interagisce con il database esclusivamente tramite le repository, senza eseguire query SQL direttamente.

Esempio di utilizzo nel backend

```
// Senza Repository Pattern
$query = "SELECT * FROM applicazione.utente WHERE email = '$email'";
$result = $db->query($query); // Vulnerabile a SQL injection!

// Con Repository Pattern
$userRepository = new UserRepository($db);
$user = $userRepository->findByEmail($email); // Sicuro, astratto
```

Test

La verifica delle funzionalità principali avviene tramite una piccola suite di *test manuali* implementati come pagine PHP, accessibili da browser, che eseguono controlli di tipo **smoke test** e **integration test** su database e repository.

Struttura

I file di test sono collocati in `src/server/test/`:

- `index.php`: pagina indice con link ai test disponibili;
- `db_test.php`: test di connessione a PostgreSQL e lettura dati;
- `db_repo_test.php`: test delle repository (CRUD).

Esecuzione

Per effettuare i test è necessario avere installato Docker Compose version 5.0.1 e un browser. Dopo l'avvio dell'ambiente Docker (servizi backend e database), i test vengono eseguiti aprendo la pagina `http://localhost:8080/test/index.php` dal browser e selezionando il test desiderato.

Test database (db_test.php)

Questo test verifica:

- la corretta configurazione della connessione (creazione PDO);
- la raggiungibilità del server PostgreSQL (utente, versione);
- l'esecuzione di una query di lettura su una tabella applicativa.

L'output mostra informazioni sulla connessione e un esempio di risultato in formato JSON.

Test repository (db_repo_test.php)

Questo test valida l'integrazione tra backend e database tramite il livello di repository. In particolare:

- inizializza la connessione (tramite DatabasePDO e DatabaseWrapper);
- istanzia una repository (es. UtenteRepository);
- esegue operazioni rappresentative (creazione, ricerca, ordinamento, aggiornamento, conteggio);
- restituisce l'esito in formato JSON.

Le query risultano incapsulate nelle repository e non sono eseguite direttamente dai livelli superiori dell'applicazione.